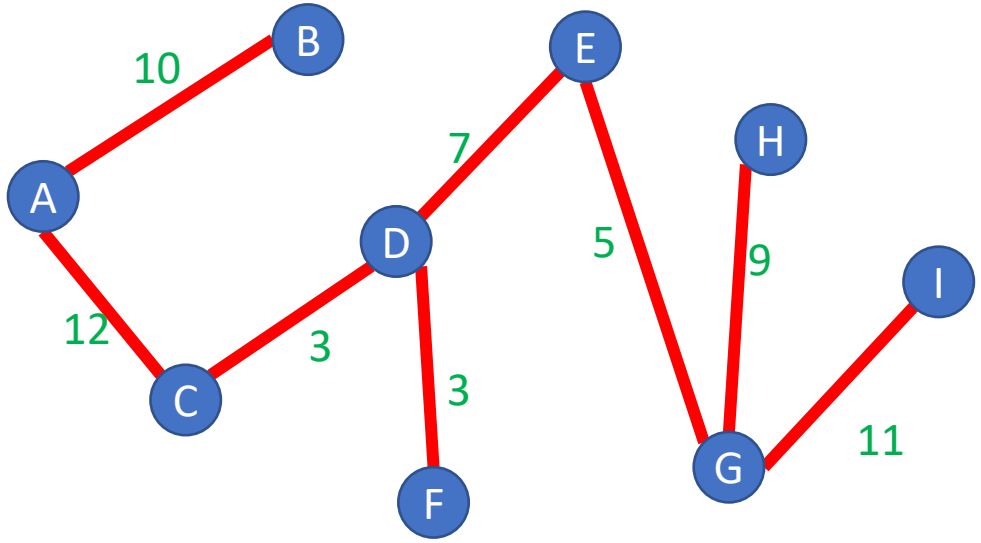
CSE 332 Autumn 2024
Lecture 26: Minimum Spanning Trees

Nathan Brunelle

http://www.cs.uw.edu/332

Definition: Tree

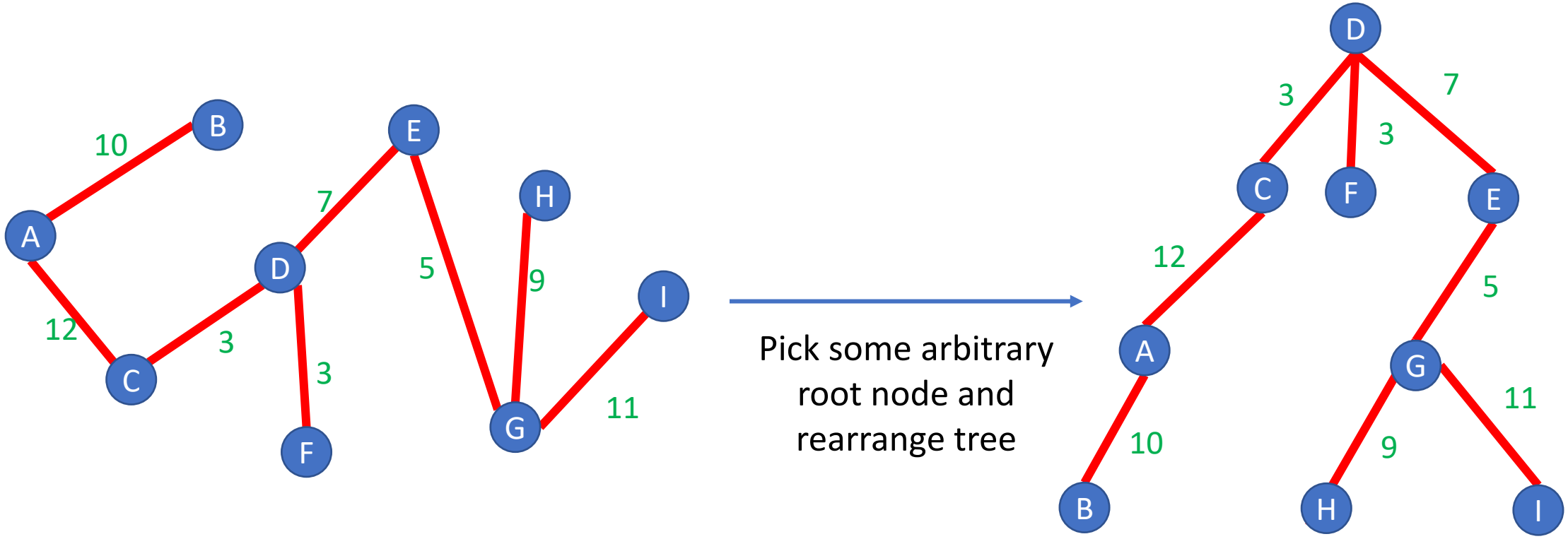
A connected graph with no cycles



Note: A tree does not need a root, but they often do!

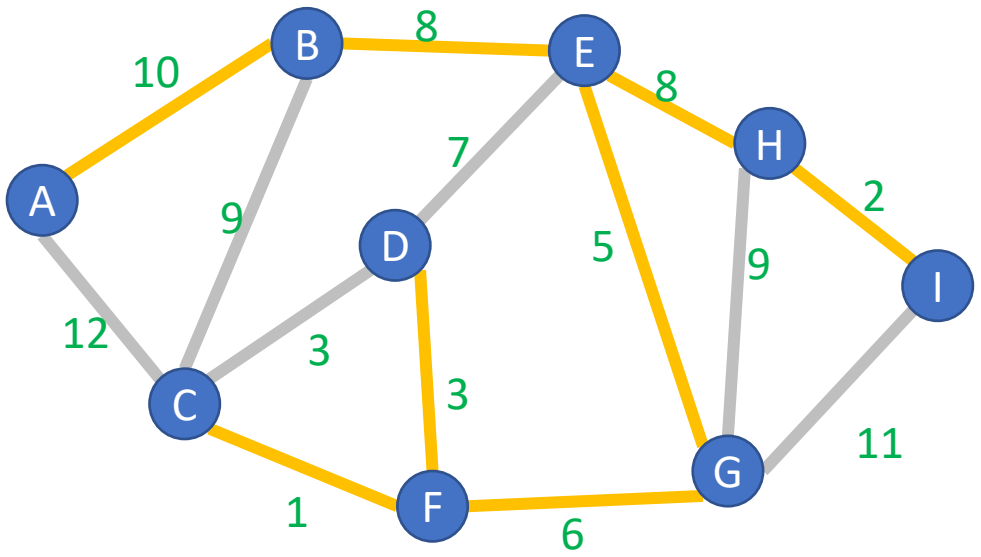
Definition: Tree

A connected graph with no cycles



Definition: Spanning Tree

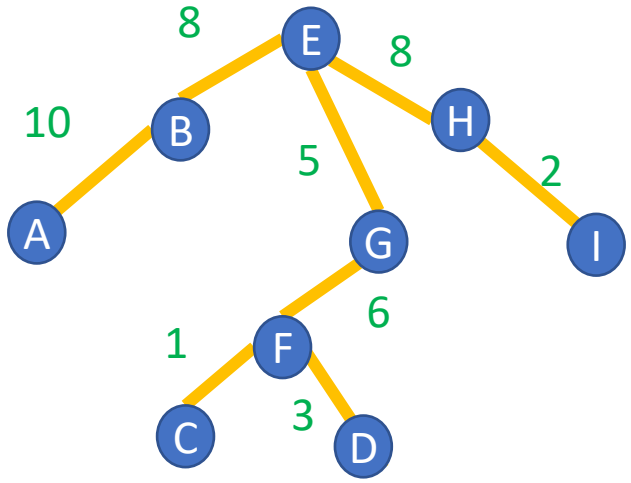
A Tree $\boldsymbol{T} = (V_T, E_T)$ which connects ("spans") all the nodes in a graph $G = (V, E)$



How many edges does T have?

$V - 1$

Pick some arbitrary root node and rearrange tree

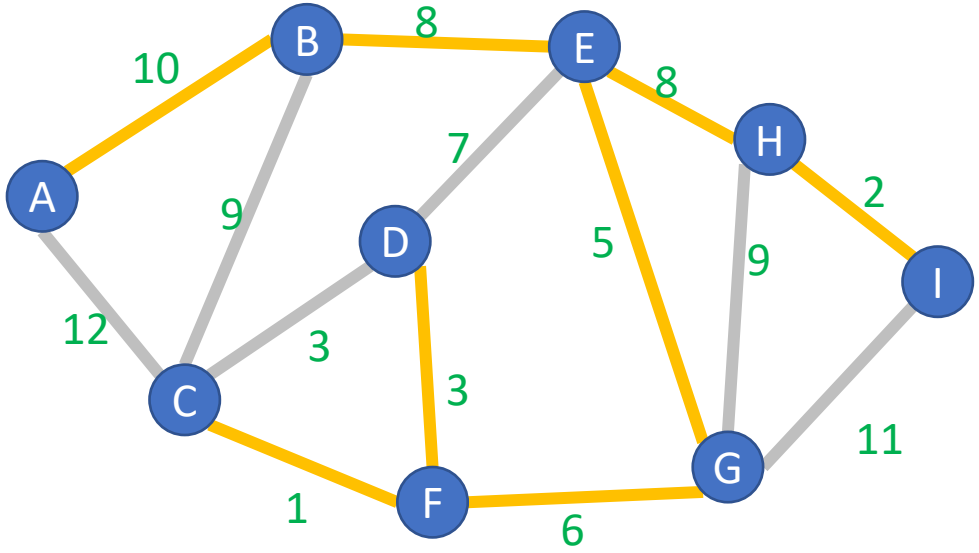


Any set of V-1 edges in the graph that doesn't have any cycles is guaranteed to be a spanning tree!

Any set of V-1 edges that connects all the nodes in the graph is guaranteed to be a spanning tree!

Definition: Minimum Spanning Tree

A Tree $\boldsymbol{T} = (V_T, E_T)$ which connects ("spans") all the nodes in a graph $G = (V, E)$, that has minimal cost



$$Cost(T) = \sum_{e \in E_T} w(e)$$

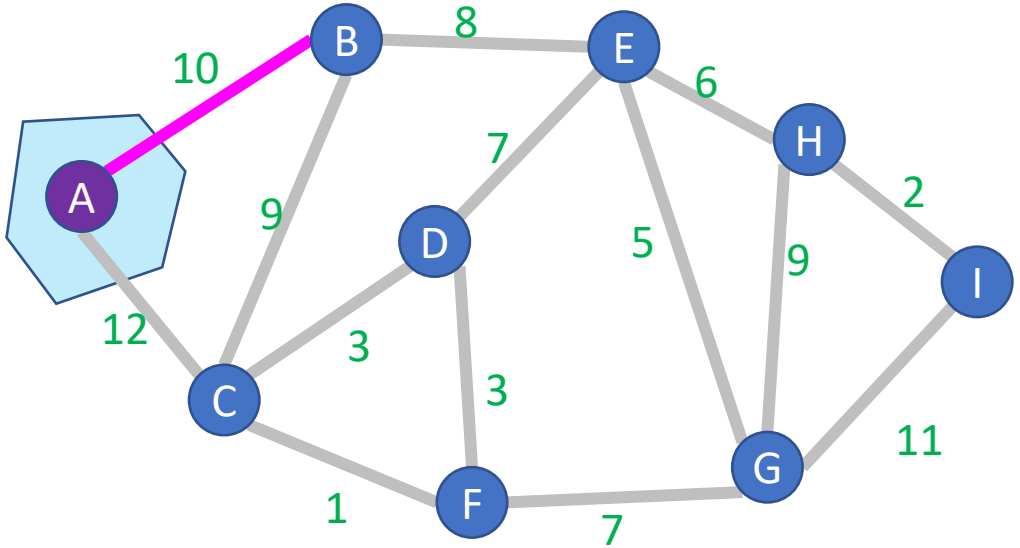
Prim's Algorithm

Start with an empty tree A

Pick a start node

Repeat $V - 1$ times:

Add the min-weight edge which connects to node in A with a node not in A



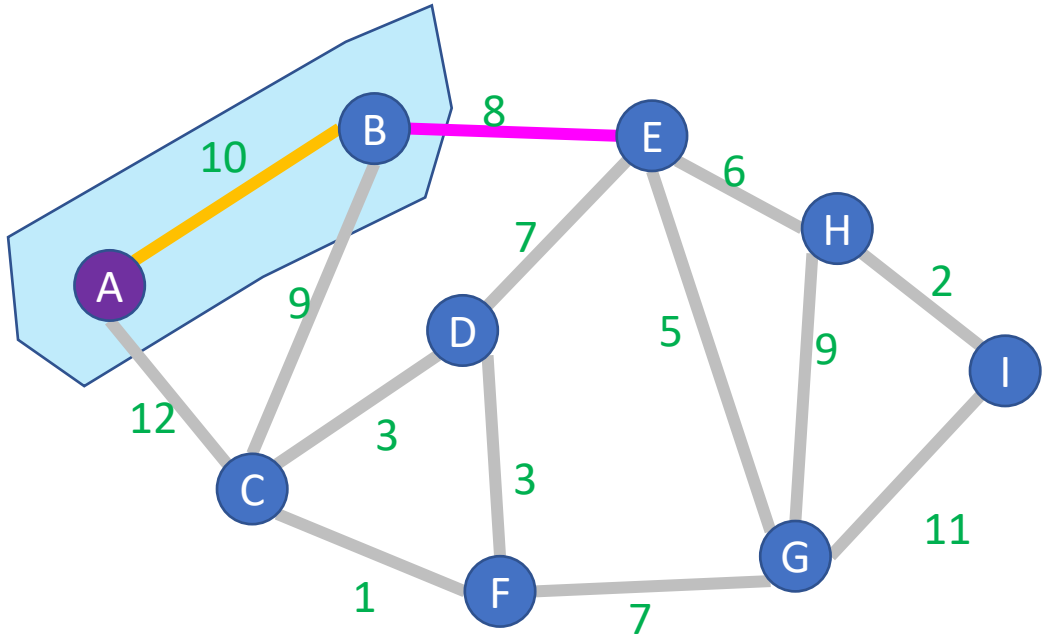
Prim's Algorithm

Start with an empty tree A

Pick a start node

Repeat $V - 1$ times:

Add the min-weight edge which connects to node in A with a node not in A



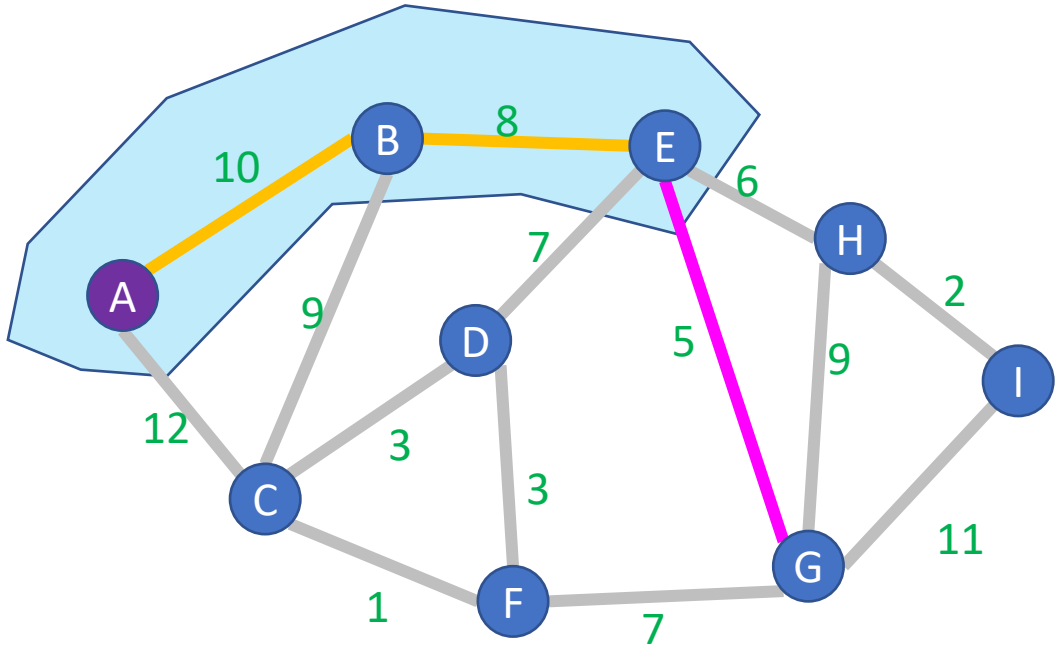
Prim's Algorithm

Start with an empty tree A

Pick a start node

Repeat $V - 1$ times:

Add the min-weight edge which connects to node in A with a node not in A



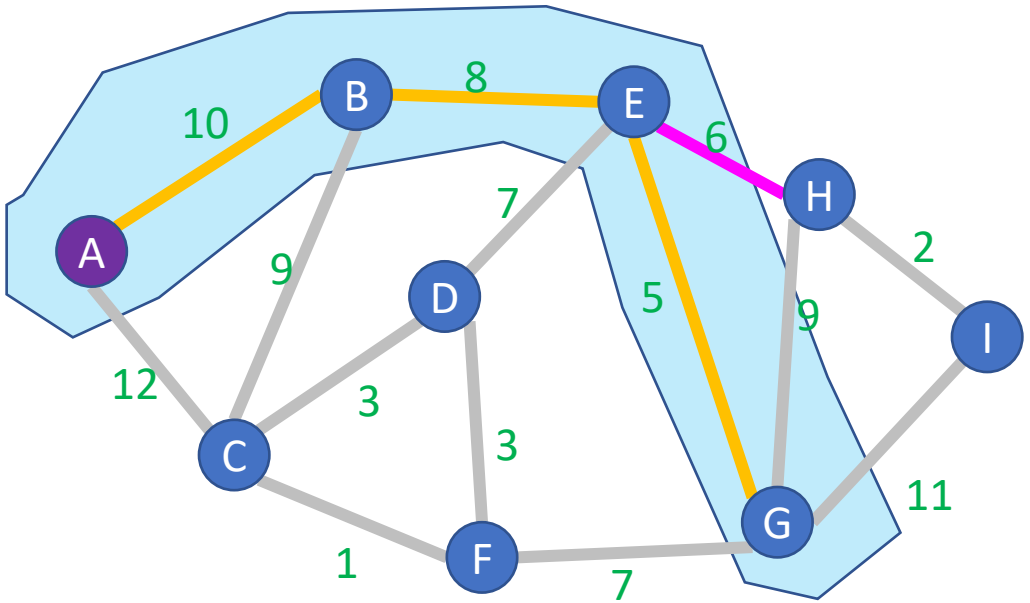
Prim's Algorithm

Start with an empty tree A

Pick a start node

Repeat $V - 1$ times:

Add the min-weight edge which connects to node in A with a node not in A



Prim's Algorithm

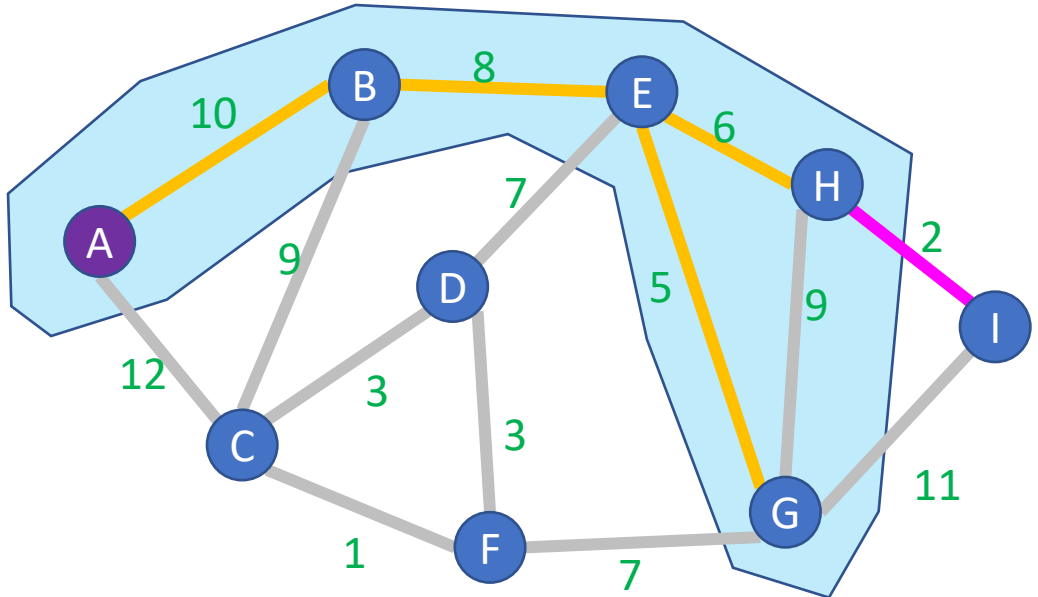
Start with an empty tree A

Pick a start node

Repeat $V - 1$ times:

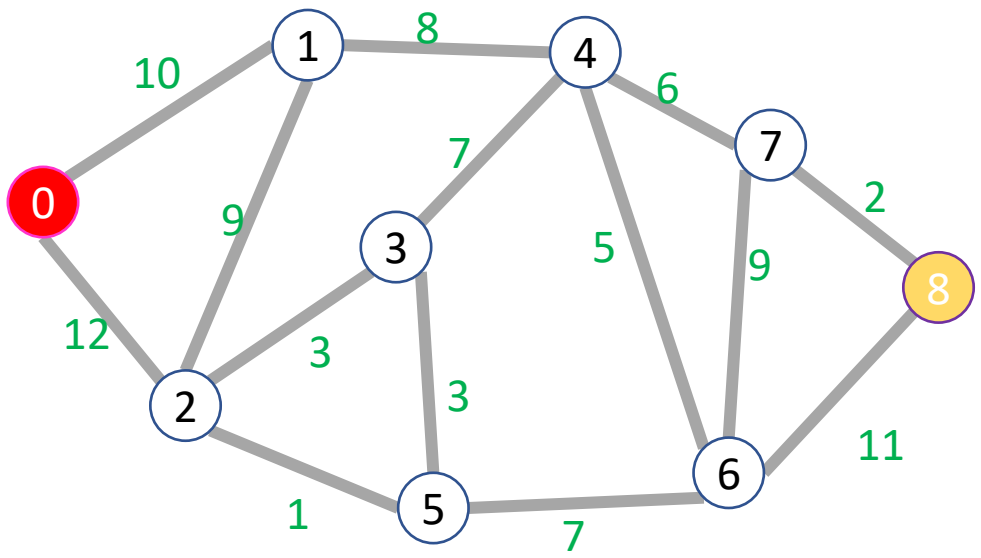
Add the min-weight edge which connects to node in A with a node not in A

Keep edges in a Heap
$O(E \log V)$



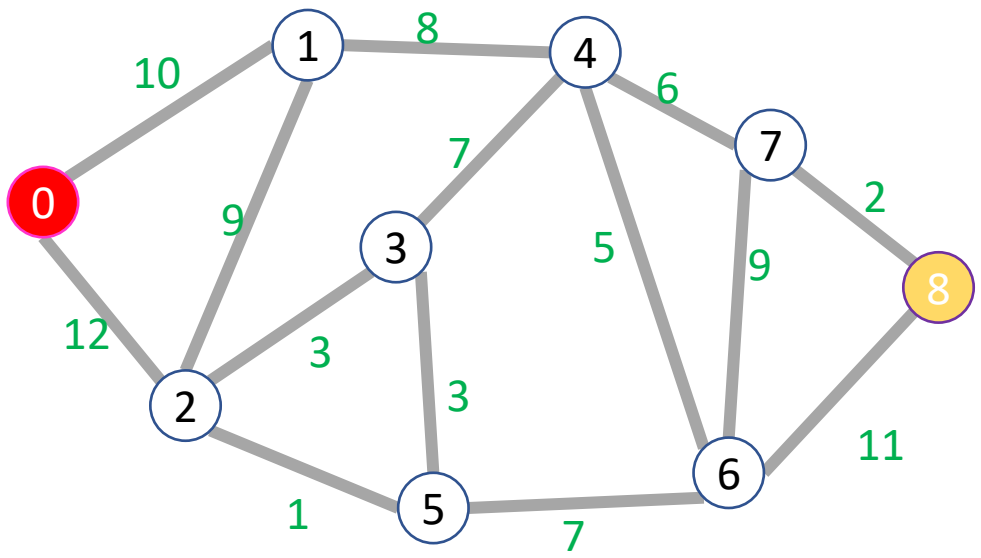
Dijkstra's Algorithm

```
Distance to start = 0
Add the start node to PQ with priority 0
Mark start as "seen"
While the PQ is not empty:
    curr = PQ.extract()
    mark curr as "done"
    add the edge (previous of curr, curr) to the spanning tree
    for each neighbor v of curr:
        d = distance to curr + weight of (curr,v)
        if v is not "seen":
            mark v as "seen"
            distance to v = d
            previous of v = curr
            PQ.add(v, d);
        if v is not "done" && d < distance to v:
            distance to v = d
            previous of v = curr
            PQ.decreaseKey(v, d)
Return the spanning tree
```



Prim's Algorithm

```
Distance to start = 0
Add the start node to PQ with priority 0
Mark start as "seen"
While the PQ is not empty:
    curr = PQ.extract()
    mark curr as "done"
    add the edge (previous of curr, curr) to the spanning tree
    for each neighbor v of curr:
        d = weight of (curr,v)
        if v is not "seen":
            mark v as "seen"
            distance to v = d
            previous of v = curr
            PQ.add(v, d);
        if v is not "done" && d < distance to v:
            distance to v = d
            previous of v = curr
            PQ.decreaseKey(v, d)
Return the spanning tree
```



Avoiding Decrease Key

- Java's `PriorityQueue` interface does not have `decreaseKey`
- Two strategies to avoid this:
  - Allow copies of nodes to be on the PQ:
    - Every time we would want to decrease the key, instead add the node again, then make sure we skip "done" nodes when extracting.
  - Store edges on the priority queue instead of nodes.
    - If the destination of the edge has already been extracted, ignore that edge. Also avoids keeping track of previous nodes.
- Both change the running time from $\Theta(E \log V)$ to $\Theta(E \log E)$
  - Since the maximum size of the priority queue is E
  - Because $E \leq V^2$, the worst case asymptotic running time does not change

Prim's Algorithm – Original Recipe

```
Distance to start = 0
Add the start node to PQ with priority 0
Mark start as "seen"
While the PQ is not empty:
   curr = PQ.extract()
   mark curr as "done"
   add the edge (previous of curr, curr) to the spanning tree
   for each neighbor v of curr:
      d = weight of (curr,v)
      if v is not "seen":
         mark v as "seen"
         distance to v = d
         previous of v = curr
         PQ.add(v, d);
      if v is not "done" && d < distance to v:
         distance to v = d
         previous of v = curr
         PQ.decreaseKey(v, d)
Return the spanning tree
```

Prim's Algorithm – Duplicate Nodes on the PQ

```
Distance to start = 0
Add the start node to PQ with priority 0
Mark start as "seen"
While the PQ is not empty:
   curr = PQ.extract()
   if(curr is "done") then continue
   mark curr as "done"
   add the edge (previous of curr, curr) to the spanning tree
   for each neighbor v of curr:
      d = weight of (curr,v)
      if v is not "seen":
         mark v as "seen"
         distance to v = d
         previous of v = curr
         PQ.add(v, d);
      if v is not "done" && d < distance to v:
         distance to v = d
         previous of v = curr
         PQ.add(v, d)
Return the spanning tree
```

Prim's Algorithm – Edges on the PQ

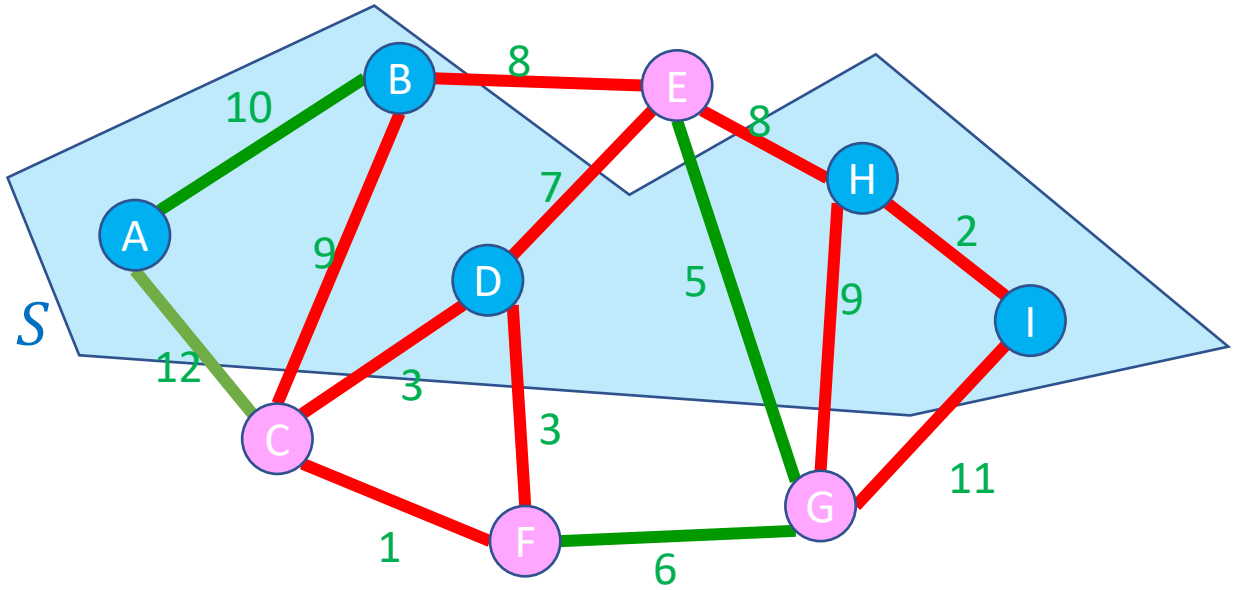
```
Distance to start = 0
Add the start node to PQ with priority 0
Mark start as "seen"
While the PQ is not empty:
   currEdge = PQ.extract()
   curr = currEdge.destination
   if(curr is "done") then continue
   mark curr as "done"
   add currEdge to the spanning tree
   for each neighbor v of curr:
      d = weight of (curr,v)
      if v is not "seen":
         mark v as "seen"
         distance to v = d
         PQ.add((curr,v),d);
      if v is not "done" && d < distance to v:
         distance to v = d
         PQ.add((curr,v), d)
Return the spanning tree
```

Why does this work?

- To argue that Prim's produces a minimum spanning tree:
  - First we show that Prim's produces a spanning tree
    - Show two of:
      - Connected
      - Acyclic
      - $V - 1$ edges
  - Then we show that it is a minimum spanning tree
    - Show all edges chosen are MST edges
      - Using the "Cut Theorem"

Definition: Cut

A Cut of graph $G = (V, E)$ is a partition of the nodes into two sets, S and $V - S$



Edge $(v_1, v_2) \in E$ crosses a cut if $v_1 \in S$ and $v_2 \in V - S$ (or opposite), e.g. (A, C)

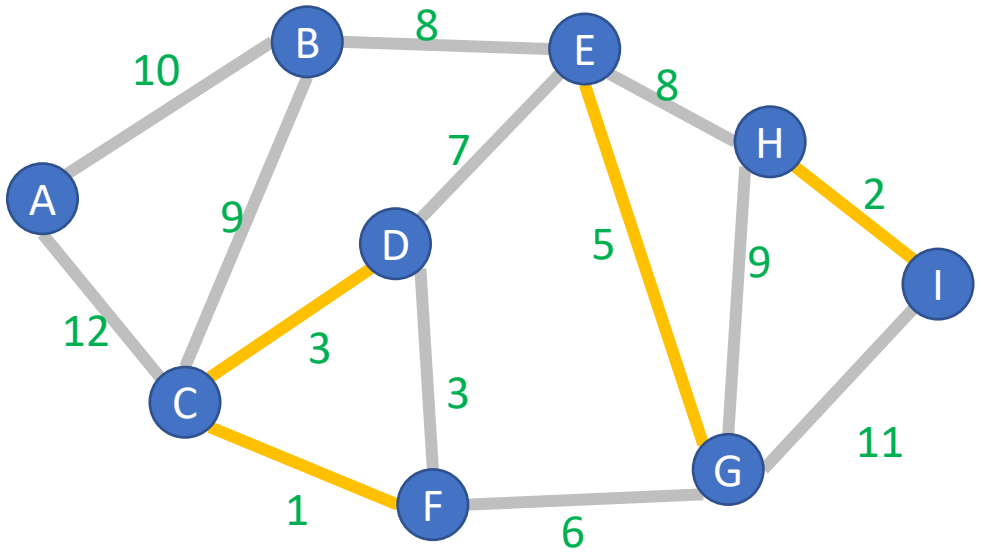
A set of edges R Respects a cut if no edges cross the cut e.g. $R = \{(A, B), (E, G), (F, G)\}$

Cut Theorem

If a set of edges A is a subset of a minimum spanning tree T, let $(S, V - S)$ be any cut which A respects. Let e be the least-weight edge which crosses $(S, V - S)$. $A \cup \{e\}$ is also a subset of a minimum spanning tree.

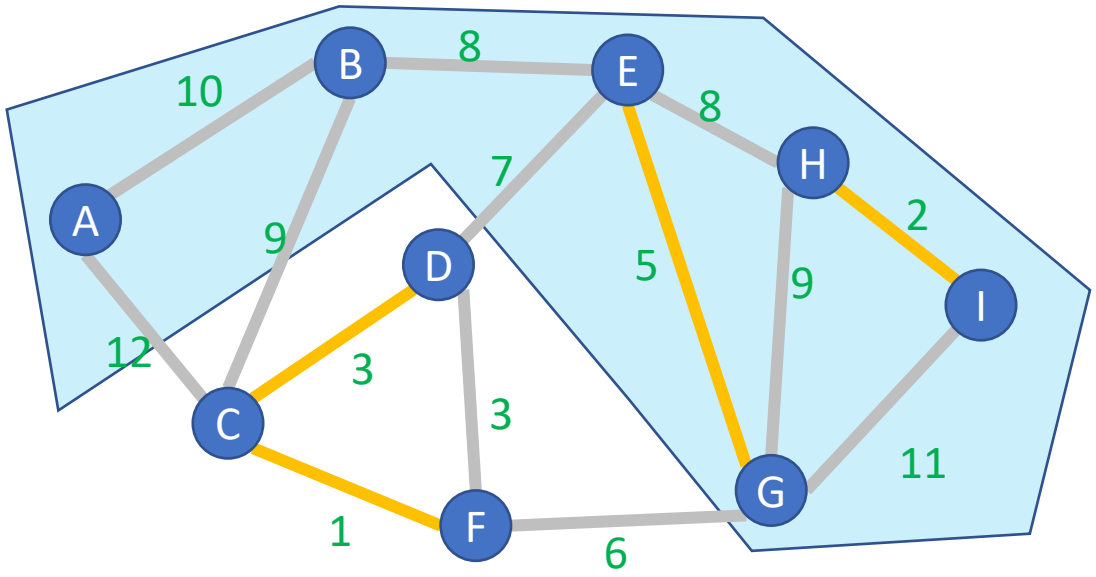
Cut Theorem

If a set of edges A is a subset of a minimum spanning tree T, let $(S, V - S)$ be any cut which A respects. Let e be the least-weight edge which crosses $(S, V - S)$. $A \cup \{e\}$ is also a subset of a minimum spanning tree.



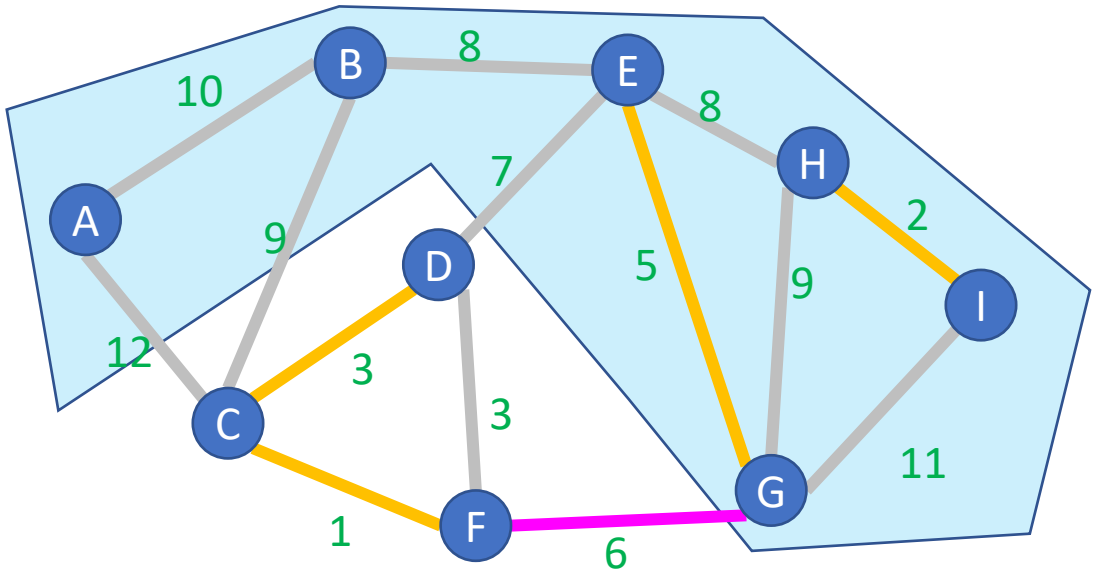
Cut Theorem

If a set of edges A is a subset of a minimum spanning tree T, let $(S, V - S)$ be any cut which A respects. Let e be the least-weight edge which crosses $(S, V - S)$. $A \cup \{e\}$ is also a subset of a minimum spanning tree.



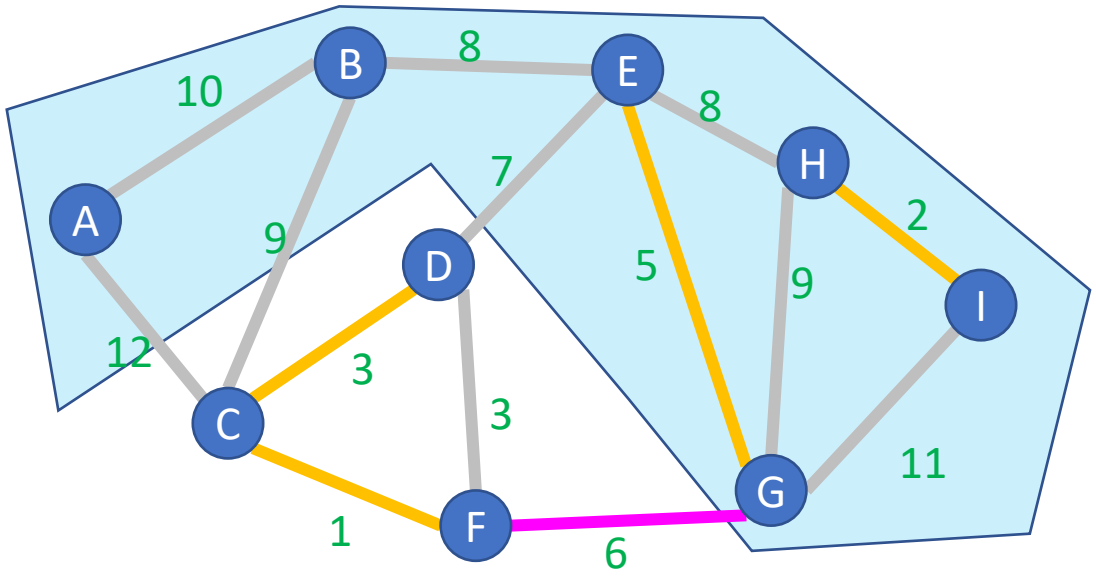
Cut Theorem

If a set of edges A is a subset of a minimum spanning tree T, let $(S, V - S)$ be any cut which A respects. Let e be the least-weight edge which crosses $(S, V - S)$. $A \cup \{e\}$ is also a subset of a minimum spanning tree.



Cut Theorem

If a set of edges A is a subset of a minimum spanning tree T, let $(S, V - S)$ be any cut which A respects. Let e be the least-weight edge which crosses $(S, V - S)$. $A \cup \{e\}$ is also a subset of a minimum spanning tree.

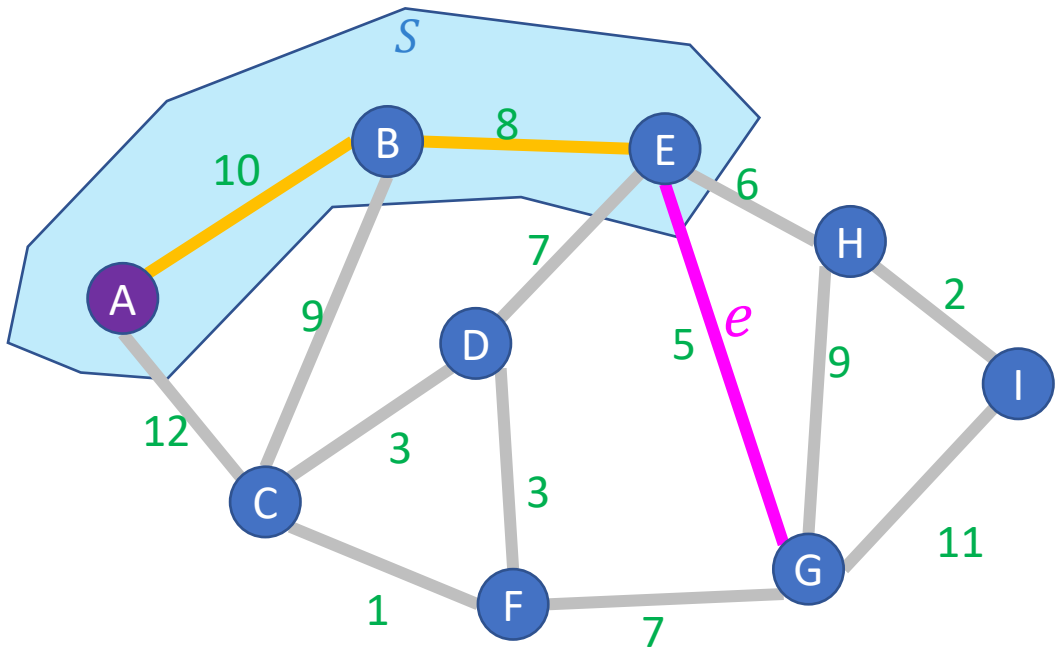


Proof of Prim's Algorithm

Start with an empty tree A

Repeat $V - 1$ times:

Add the min-weight edge that connects to a node not currently in the tree



Proof: By Structural Induction

Suppose we have some arbitrary set of edges A that Prims's has already selected to include in the MST. $e = (E, G)$ is the edge Prims's selects to add next

We know that there cannot exist a path from E to G using only edges in A because G has not been removed from the priority queue

We can cut the graph therefore into 2 disjoint sets:

- Nodes that have been removed from the priority queue
- All other nodes

e is the minimum cost edge that crosses this cut, so by the Cut Theorem, Prim's only selects MST edges!

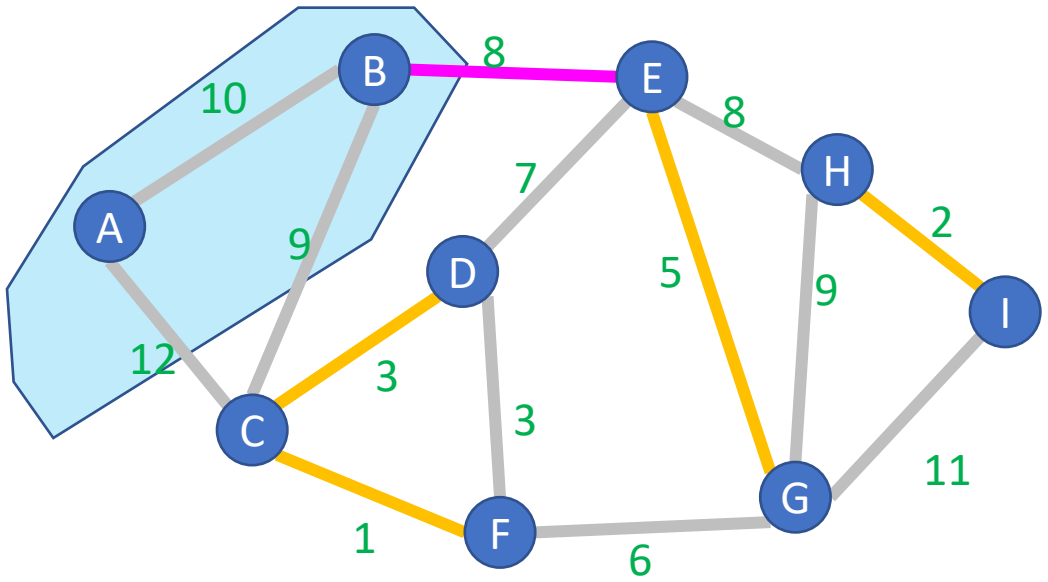
General MST Algorithm

Start with an empty tree A

Repeat $V - 1$ times:

Pick a cut $(S, V - S)$ which A respects (typically implicitly)

Add the min-weight edge which crosses $(S, V - S)$



Prim's Algorithm

Start with an empty tree A

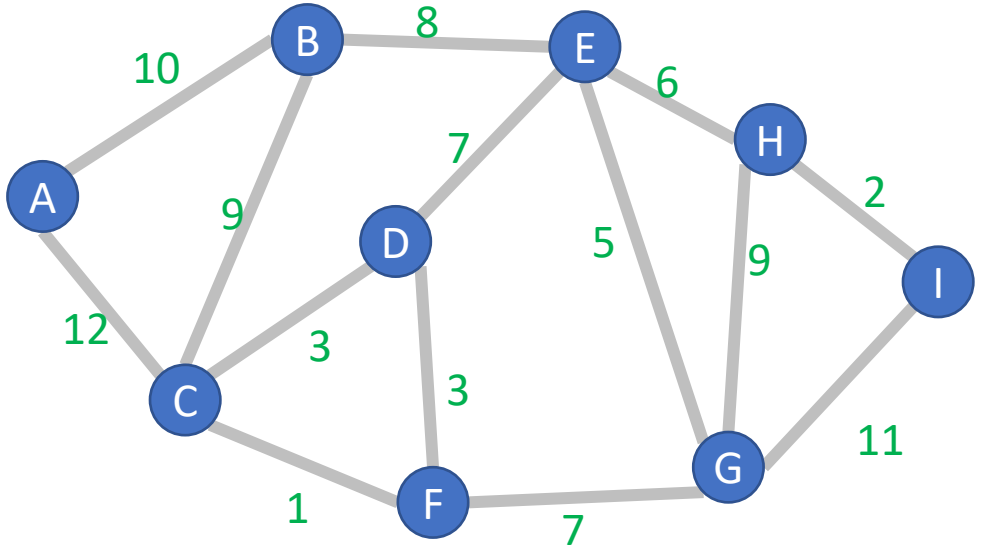
Repeat $V - 1$ times:

Pick a cut $(S, V - S)$ which A respects

Add the min-weight edge which crosses $(S, V - S)$

S is all endpoint of edges in A

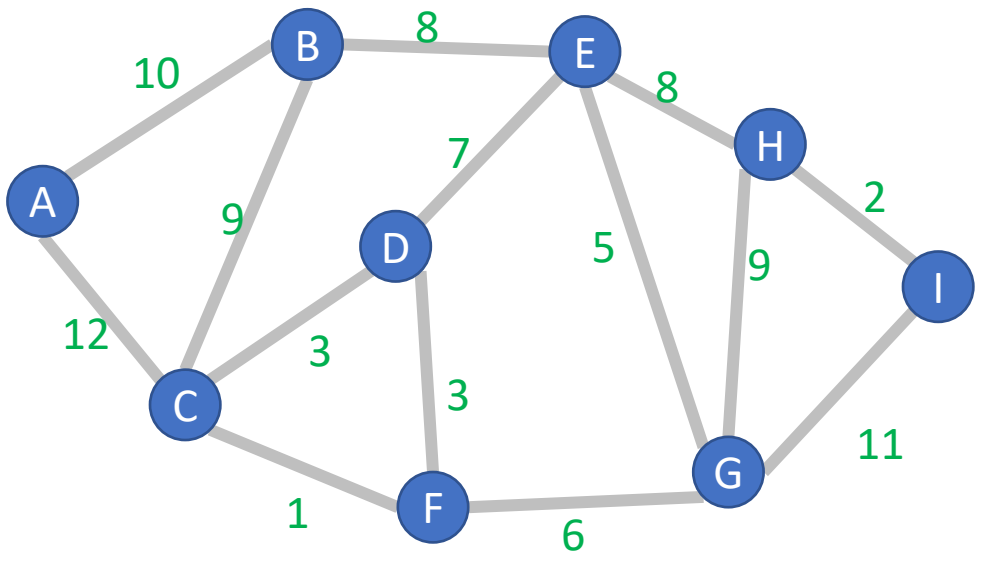
e is the min-weight edge that grows the tree



Kruskal's Algorithm

Start with an empty tree A

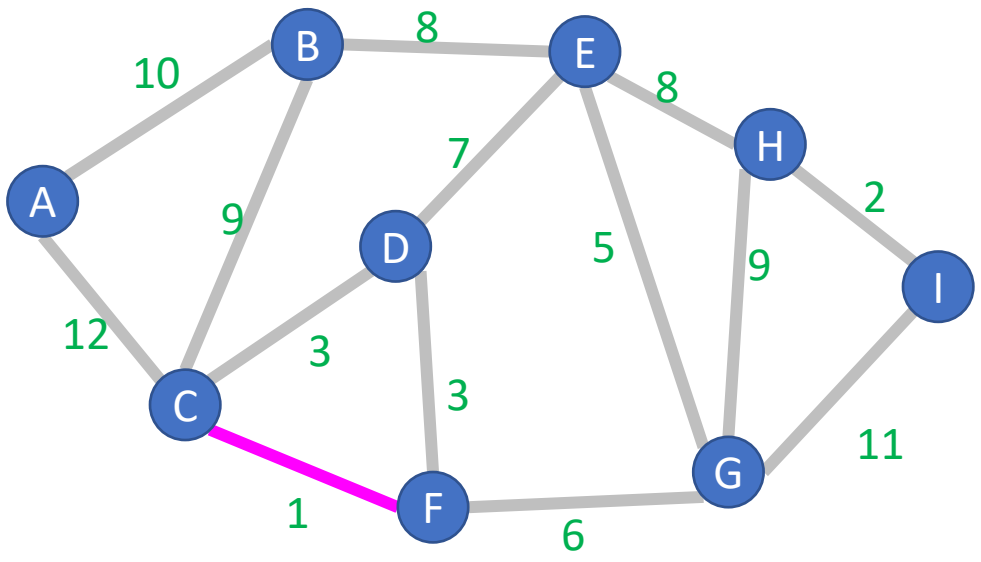
Add to A the lowest-weight edge that does not create a cycle



Kruskal's Algorithm

Start with an empty tree A

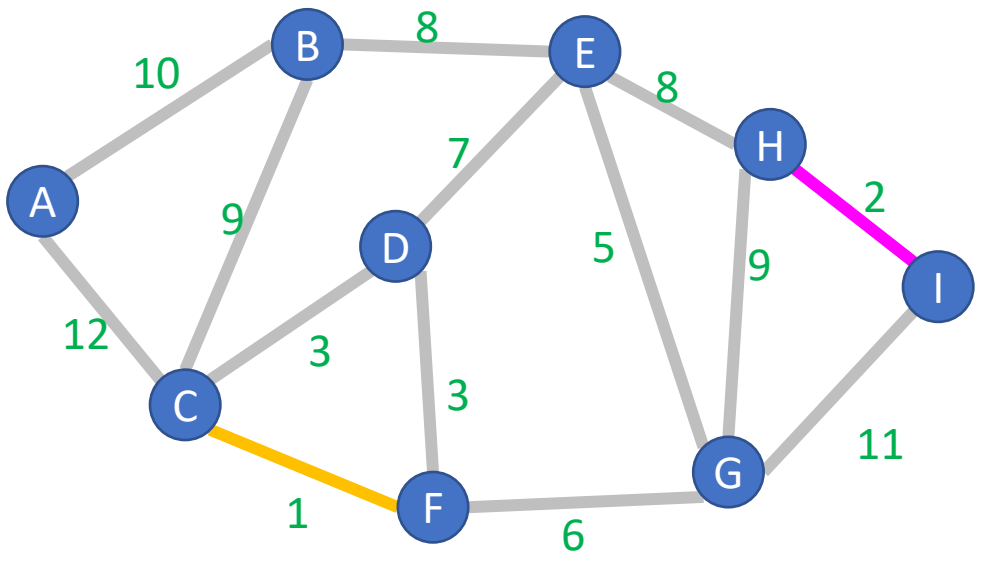
Add to A the lowest-weight edge that does not create a cycle



Kruskal's Algorithm

Start with an empty tree A

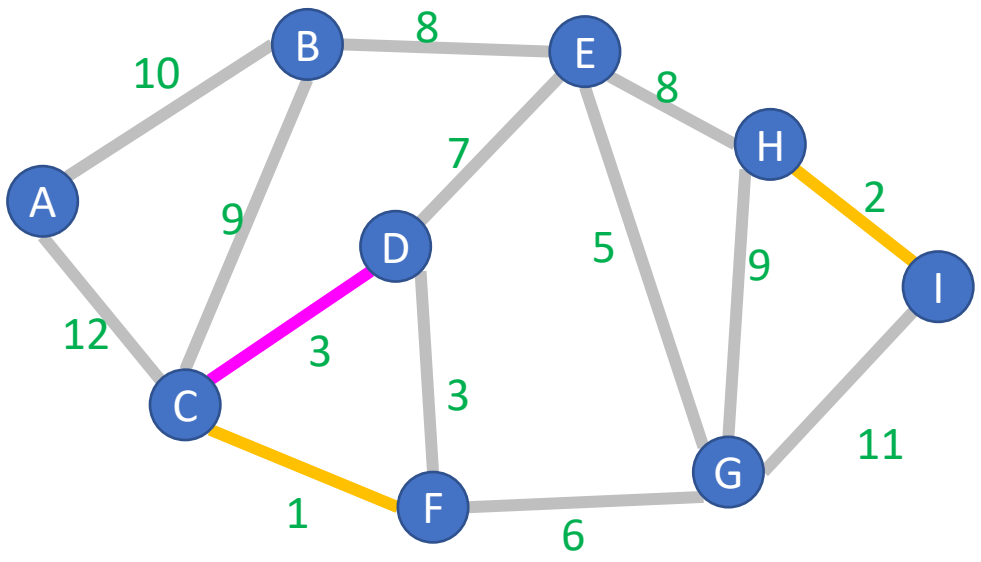
Add to A the lowest-weight edge that does not create a cycle



Kruskal's Algorithm

Start with an empty tree A

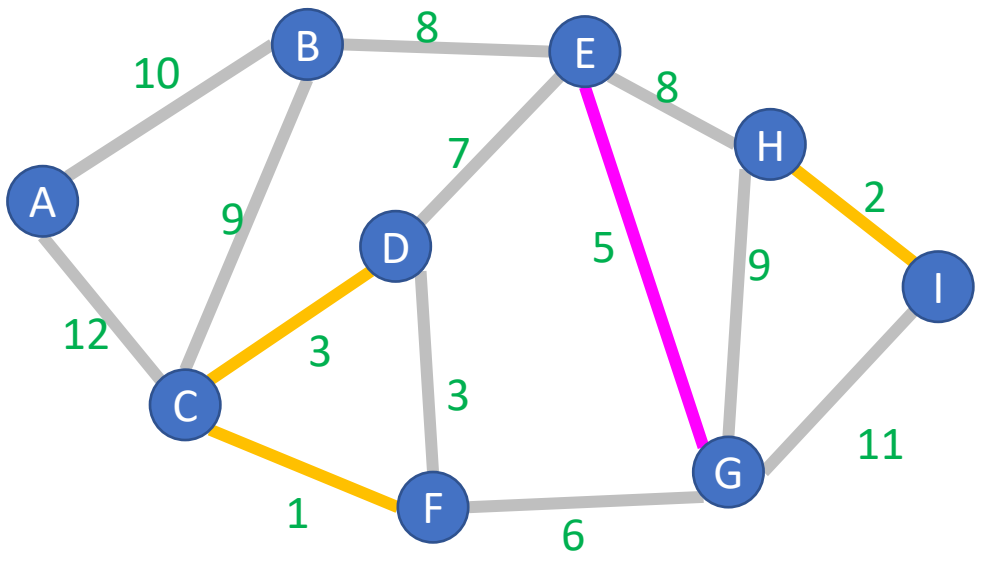
Add to A the lowest-weight edge that does not create a cycle



Kruskal's Algorithm

Start with an empty tree A

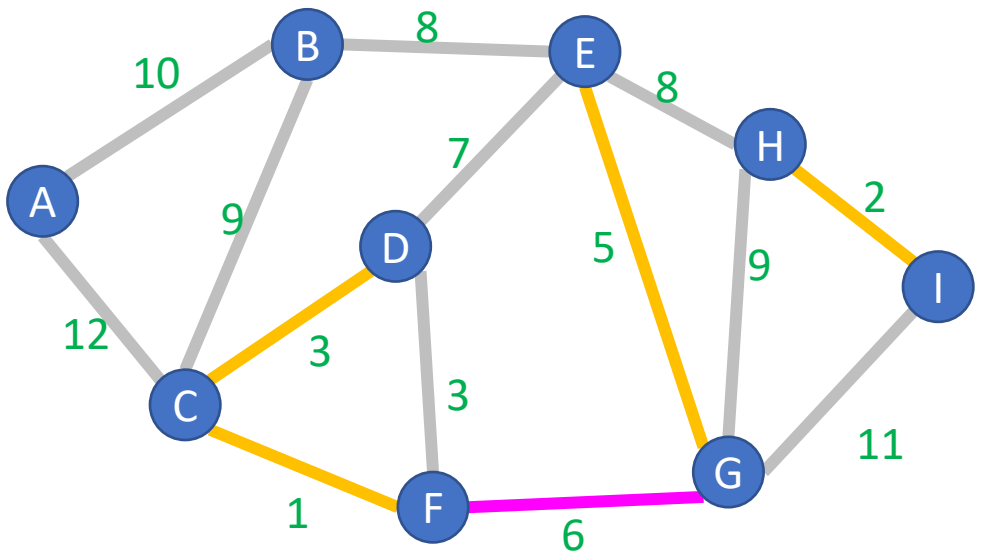
Add to A the lowest-weight edge that does not create a cycle



Kruskal's Algorithm

Start with an empty tree A

Add to A the lowest-weight edge that does not create a cycle



Correctness of Kruskal's Algorithm

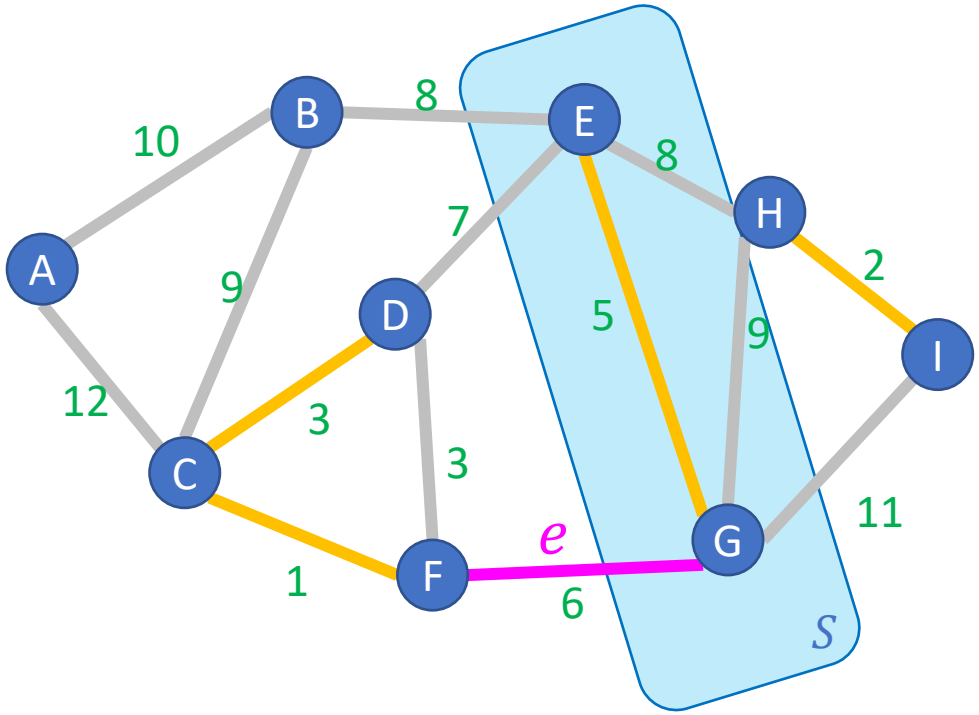
- It's sufficient to just show that it follows the template of our "General MST Algorithm"
  - Show that for every edge chosen, it is the least-weight edge which crosses some cut that respects all already-chosen edges.

Proof of Kruskal's Algorithm

Start with an empty tree A

Repeat $V - 1$ times:

Add the min-weight edge that doesn't cause a cycle



Proof: Suppose we have some arbitrary set of edges A that Kruskal's has already selected to include in the MST. $e = (F, G)$ is the edge Kruskal's selects to add next

We know that there cannot exist a path from F to G using only edges in A because e does not cause a cycle

We can cut the graph therefore into 2 disjoint sets:

- nodes reachable from G using edges in A
- All other nodes

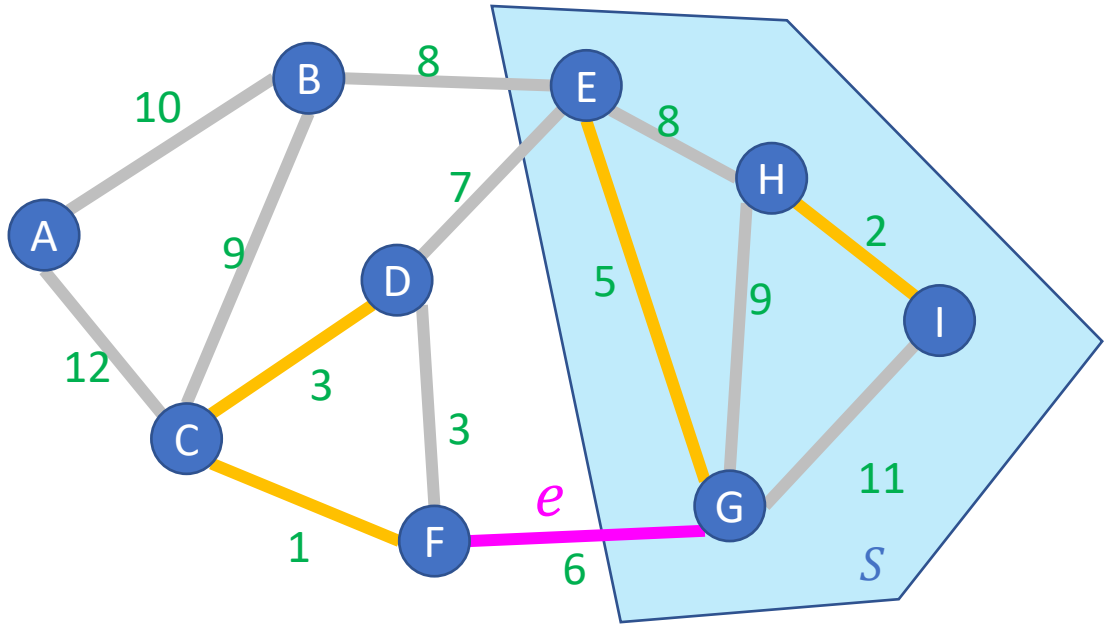
e is the minimum cost edge that crosses this cut, so by the Cut Theorem, Kruskal's is optimal!

Kruskal's Algorithm Runtime

Start with an empty tree A

Repeat $V - 1$ times:

Add the min-weight edge that doesn't cause a cycle



Keep edges in a Disjoint-set data structure (very fancy)

$O(E \log V)$